Residential Property Price Prediction Using Machine Learning

Ames Housing Dataset

Kavya Katal Aabir Sarkar Kushagra Maheswari Pratyush Chouksey

March 2, 2026

Abstract

Accurately estimating residential property prices is critical for home buyers, sellers, real-estate agents, and financial institutions. In this project, we build an end–to–end machine learning system that predicts house sale prices in USD using the Ames Housing dataset. We design a robust preprocessing pipeline for mixed numeric and categorical data, apply a log-transform to stabilise the target distribution, and compare a baseline Linear Regression model against a more complex Random Forest Regressor. The final model is deployed as an interactive Streamlit web application that allows users to input key house features and obtain real-time price estimates. This report documents the problem definition, data description, exploratory data analysis (EDA), modelling methodology, evaluation, optimisation steps, and the contribution of each team member.

Contents

1 Problem Statement **3**

2 Data Description **3**

2.1 Dataset Origin 3

2.2 Feature Overview 3

2.3 Target Variable 4

3 EDA Process **4**

3.1 Dataset Audit and Structure 5

3.2 Missing Values and Data Quality 5

3.3 Target Distribution and Transformation 5

3.4 Correlation Analysis 6

3.5 Train–Validation Split 6

4 Methodology **7**

4.1 Overall Pipeline Design 7

4.2 Preprocessing for Numeric Features 7

4.3 Preprocessing for Categorical Features 8

4.4 Target Transformation 8

4.5 Baseline Model: Linear Regression . . . 8
4.6 Comparison Model: Random Forest Regressor . . . 9
4.7 Model Serialization and Deployment Artifacts . . . 9

5 Evaluation **10**
5.1 Metrics . . . 10
5.2 Quantitative Results . . . 10
5.3 Interpretation . . . 10
5.4 Model Selection . . . 11

6 Optimization and Design Choices **11**
6.1 Data Cleaning and Preprocessing . . . 11
6.2 Target Transformation . . . 11
6.3 Model Architecture and Hyperparameters . . . 11
6.4 Feature Selection and UI Design . . . 11
6.5 Deployment Optimizations . . . 12

7 Streamlit Application **12**
7.1 User Interface . . . 12
7.2 Prediction Workflow . . . 13

8 Team Contribution **13**

9 Conclusion **14**

1 Problem Statement

Buying or selling a house involves high-stakes financial decisions. However, manually estimating a fair market value is difficult because property prices depend on a large number of factors such as neighbourhood, living area, construction quality, age, and amenities. Traditional appraisal methods are labour-intensive, subjective, and hard to scale.

Goal: Given structured information about a residential property, we aim to automatically predict its sale price (in USD) using supervised machine learning. The model should:

- Produce **reasonably accurate and stable** price estimates on unseen properties.
- **Generalise** beyond the training data without overfitting.
- Be **fast and lightweight** enough to support an interactive web interface.
- Be **interpretable and maintainable**, with a clear preprocessing pipeline and reproducible training process.

Scope: The project focuses on predicting prices for properties in the Ames, Iowa housing market using historical sales data. While the deployed Streamlit app exposes only a subset of features to the user (e.g., overall quality, living area, basement area, year built), the underlying training pipeline handles the full feature set of the dataset.

2 Data Description

2.1 Dataset Origin

We use the **Ames Housing** dataset, popularised via the Kaggle competition "House Prices: Advanced Regression Techniques". The dataset contains detailed information about residential property sales in Ames, Iowa.

- **Source:** Kaggle / Ames Housing public dataset.
- **File used:** A large tabular CSV (referred to as `BigHousing.csv` in this repository).
- **Target variable:** `SalePrice` (USD).

2.2 Feature Overview

The raw dataset contains around 80 input features describing each property. These features fall into the following broad categories:

- **Physical characteristics**
  - `GrLivArea`: Above-ground living area (square feet).
  - `TotalBsmtSF`: Basement area (square feet).
  - `LotArea`: Lot size (square feet).

  - `BsmtFullBath`, `FullBath`: Number of bathrooms.
  - `GarageCars`: Garage capacity (number of cars).
  - `Fireplaces`: Number of fireplaces.

- **Quality and condition**
  - `OverallQual`: Overall material and finish quality (1–10).
  - `OverallCond`: Overall condition rating.
  - Exterior/interior material quality ratings (e.g. `ExterQual`, `KitchenQual`).

- **Age and year-related variables**
  - `YearBuilt`: Original construction date.
  - `YearRemodAdd`: Remodel date.
  - `YrSold`: Year sold.

- **Location and configuration**
  - `Neighborhood`: Physical location within Ames.
  - `MSZoning`: General zoning classification.
  - `HouseStyle`, `BldgType`: Building and house style.

In our training pipeline, we automatically infer column types:

- **36 numeric** columns (floats/integers).
- **43 categorical** columns (strings or categories).

The `Id` column is an identifier and is dropped before training.

2.3 Target Variable

The target is the final sale price of each property:

- **Name:** `SalePrice`
- **Unit:** USD
- **Type:** Continuous numeric

As explored in Section 3.3, `SalePrice` is right-skewed, motivating a log-transformation during model training.

3 EDA Process

Our exploratory data analysis (EDA) was performed primarily in a Jupyter notebook (`final-genai-notebook.`
and guided the preprocessing and modelling choices.

3.1 Dataset Audit and Structure

We first examined the basic structure of the dataset:

- Checked the shape: approximately 1460 rows and 80+ features.
- Inspected column data types to separate numeric and categorical variables.
- Verified that the `SalePrice` column (target) exists and has no missing values.
- Confirmed the presence of both continuous measurements (e.g., square footage) and many categorical descriptors (e.g., neighbourhood, exterior material).

3.2 Missing Values and Data Quality

We computed the count of missing values per column and sorted them in descending order. Several features exhibited substantial missingness, especially optional attributes such as `Alley`, `PoolQC`, and some basement/garage features.

Figure 1 (generated during experimentation) visualises the top 20 features with the highest number of missing values, highlighting where imputation strategies would be required.

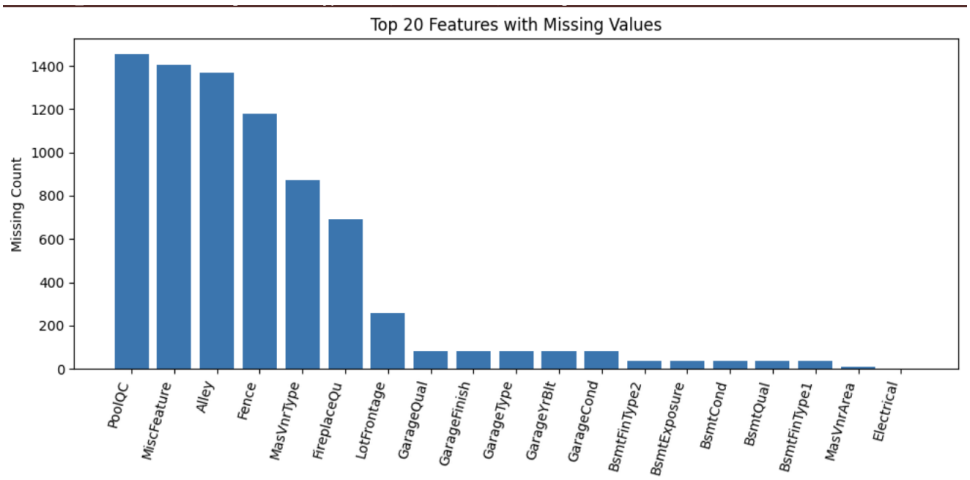

Figure 1: Top 20 features with the highest number of missing values.

3.3 Target Distribution and Transformation

The raw distribution of `SalePrice` is strongly right-skewed, with a long tail of expensive properties. This violates the normality assumption underlying many regression models and can cause models to underperform on typical houses or be overly influenced by outliers.

Using the notebook, we visualised the histogram of `SalePrice` and then the histogram of $\log(1 + \texttt{SalePrice})$. These are saved as:

- `images/salePrice_before_log1p.png`
- `images/salePrice_after_log1p.png`

The log-transformed target is much closer to a symmetric, approximately normal distribution, which stabilises training and improves model fit.

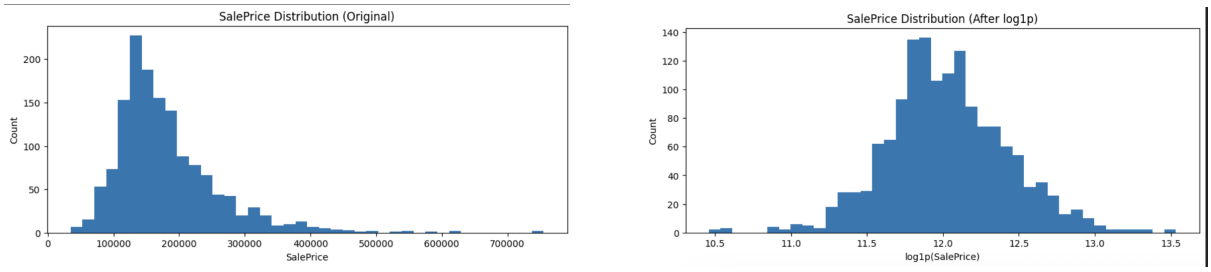

Figure 2: Distribution of `SalePrice` before (left) and after (right) $\log(1 + x)$ transformation.

3.4 Correlation Analysis

We computed the Pearson correlation between numeric features and the target (in USD) and plotted the 12 most correlated numeric features. This plot is saved as `images/Top12_Numeric_Features_Correl`

Key observations:

- **Overall quality** (`OverallQual`) has one of the strongest positive correlations with `SalePrice`.
- **Living area** (`GrLivArea`) and **basement area** (`TotalBsmtSF`) are also highly correlated.
- **Garage capacity** (`GarageCars`) and **year built** (`YearBuilt`) show meaningful positive correlations.

These insights motivated both the choice of input features highlighted in the Streamlit UI and the expectation that a mostly linear model might perform well after appropriate transformations.

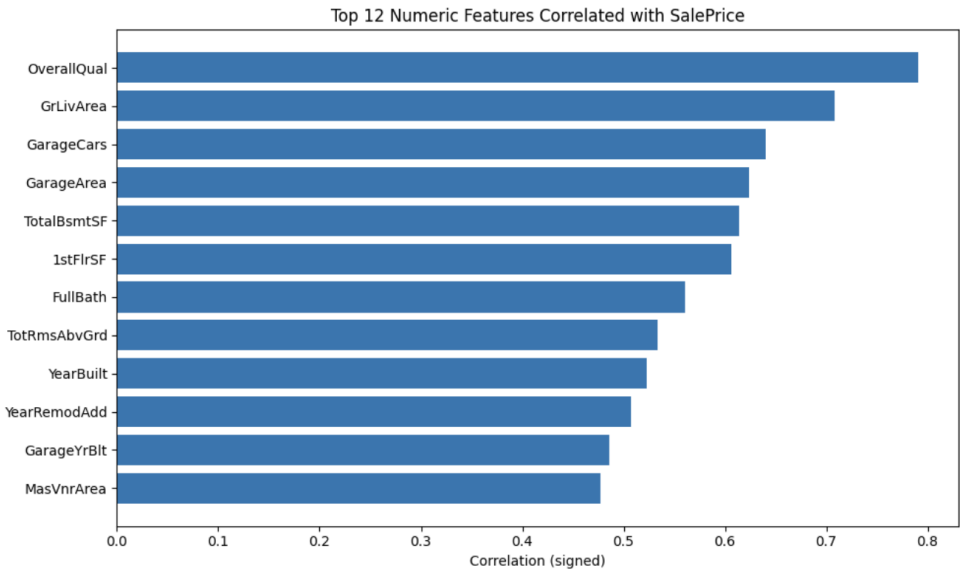

Figure 3: Top 12 numeric features by absolute correlation with `SalePrice`.

3.5 Train–Validation Split

To obtain unbiased estimates of model performance, we split the dataset into training and validation sets:

- **Train:** 80% of the data.
- **Validation:** 20% of the data.
- **Random seed:** 42 (for reproducibility).

This split results in 1168 training examples and 292 validation examples. The same split is used for all model comparisons to ensure fairness.

4 Methodology

4.1 Overall Pipeline Design

Instead of manually cleaning and transforming features in an ad hoc manner, we use a **scikit-learn pipeline** based on `ColumnTransformer` and `Pipeline` objects. This design has several advantages:

- Training and inference (in the Streamlit app) run the **exact same** preprocessing steps.
- Complex transformations are encapsulated and reproducible.
- The final model artifact can be serialised with joblib and loaded directly in production.

The high-level flow is:

1. Load and clean raw data.
2. Separate features $\mathbf{X}$ and target $y =$ `SalePrice`.
3. Drop the `Id` column.
4. Split into training and validation sets.
5. Apply $\log(1 + y)$ transformation to the target for training.
6. Fit preprocessing + model pipelines on training data.
7. Evaluate on the validation set, converting predictions back to USD using $\exp(y_{\log}) - 1$.
8. Save the final pipeline, target transform flag, and training column names as a joblib artifact.

4.2 Preprocessing for Numeric Features

Numeric features are processed through the following steps:

- **Missing value imputation:** Replace missing values with the median of each column using `SimpleImputer(strategy="median")`.
- **Scaling (for Linear Regression):** Standardise features to zero mean and unit variance using `StandardScaler`.

For the Random Forest model, scaling is not required, so we only perform median imputation.

4.3 Preprocessing for Categorical Features

Categorical features undergo:

- **Most-frequent imputation:** Fill missing values with the most frequent category per column.
- **One-hot encoding:** Convert categorical values into binary indicator columns using `OneHotEncoder(handle`

Using `handle_unknown="ignore"` ensures that the model can safely process categories that did not appear during training (important in deployment).

4.4 Target Transformation

As established in Section 3.3, the target `SalePrice` is right-skewed. We therefore train models on:

$$y_{\log} = \log(1 + \texttt{SalePrice})$$

At prediction time, we invert this transformation to obtain prices in USD:

$$\widehat{\texttt{SalePrice}} = \exp(y_{\log}^{\text{pred}}) - 1$$

This transformation:

- Reduces the influence of extreme outliers.
- Brings the target distribution closer to normal.
- Often improves both MAE and RMSE for regression tasks with heavy-tailed targets.

The transformation flag (`"log1p"`) is stored with the saved artifact so the Streamlit app can consistently apply the inverse transform via `numpy.expm1`.

4.5 Baseline Model: Linear Regression

We first build a baseline using ordinary least squares Linear Regression:

- **Model:** scikit-learn's `LinearRegression`.
- **Pipeline:**
  - Numeric: median imputation + standard scaling.
  - Categorical: most-frequent imputation + one-hot encoding.
  - Regressor: Linear Regression trained on $y_{\log}$.

The Linear Regression pipeline is simple, fast, and provides a strong baseline for comparison.

4.6 Comparison Model: Random Forest Regressor

To capture potential non-linear relationships and interactions, we also train a Random Forest Regressor:

- **Model:** `RandomForestRegressor` with:
  - `n_estimators = 600`
  - `random_state = 42`
  - `n_jobs = -1` (parallel training)
  - Other parameters set to defaults (e.g. `max_depth=None`).
- **Pipeline:**
  - Numeric: median imputation (no scaling).
  - Categorical: most-frequent imputation + one-hot encoding.
  - Regressor: Random Forest trained on $y_{\log}$.

Random Forest can automatically model non-linearities and feature interactions, although at the cost of greater complexity and less interpretability compared to Linear Regression.

4.7 Model Serialization and Deployment Artifacts

After training, we save the artifacts using joblib:

- **Pipeline:** The final trained Linear Regression pipeline (preprocessing + model).
- **Target transform flag:** `"log1p"` to indicate the target was transformed.
- **Training columns:** The list of original feature columns expected by the pipeline.

These are stored in a single joblib file (`model/house_price_lr_pipeline.joblib`) which is loaded by the Streamlit app.

The deployed application uses:

- `load_artifacts`**:** Loads the saved pipeline, target transform, and training columns.
- `build_input_df`**:** Constructs a single-row `pandas.DataFrame` matching the training feature schema.
- `predict_usd`**:** Runs the pipeline, applies the inverse transform if required, validates the prediction, and returns a final price in USD.

5 Evaluation

5.1 Metrics

We evaluate regression performance on the validation set using:

- **MAE (Mean Absolute Error):**

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

  Measures the average absolute deviation in USD.

- **RMSE (Root Mean Squared Error):**

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

  Penalises larger errors more heavily than MAE.

- **$\mathbf{R^2}$ (Coefficient of Determination):**

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

  Measures the proportion of variance in the target explained by the model.

All metrics are reported in the original price space (USD), after inverting the log-transform.

5.2 Quantitative Results

Table 1 summarises the performance of both models on the validation set.

Table 1: Validation performance of Linear Regression and Random Forest models.

Model	**MAE (USD)**	**RMSE (USD)**	$\mathbf{R^2}$
Linear Regression (log target)	14,899	22,740	0.9326
Random Forest (log target)	17,495	30,153	0.8815

5.3 Interpretation

Despite being simpler, the Linear Regression model:

- Achieves a **lower MAE and RMSE** compared to the Random Forest.
- Explains a **higher proportion of variance** in prices ($R^2 \approx 0.93$ vs. 0.88).

These results suggest that, after appropriate preprocessing and log-transforming the target, the relationship between the selected features and `SalePrice` is largely linear. The more complex Random Forest model underperforms, likely because it overfits some noise and does not benefit enough from modelling non-linearities.

5.4 Model Selection

Based on the validation metrics and practical concerns (speed, interpretability, and ease of deployment), we select the **Linear Regression (log target)** pipeline as the final model for deployment.

6 Optimization and Design Choices

This section summarises the key optimisation steps taken to improve model performance and deployment readiness.

6.1 Data Cleaning and Preprocessing

- **Identifier removal:** The `Id` column, which does not carry predictive information, was removed.
- **Missing value handling:** We used median imputation for numeric columns and most-frequent imputation for categorical columns.
- **Consistent pipeline:** All preprocessing was encapsulated in scikit-learn pipelines, ensuring that training and inference apply identical transformations.

6.2 Target Transformation

Using a $\log(1 + x)$ target transformation improved the stability of training and reduced the influence of extremely expensive properties. This was one of the most impactful optimisations and directly improved both MAE and R^2.

6.3 Model Architecture and Hyperparameters

- **Baseline vs. complex models:** We intentionally kept a simple, well-regularised Linear Regression baseline for comparison.
- **Random Forest tuning:**
  - Increased the number of trees (`n_estimators = 600`) to stabilise the ensemble.
  - Allowed deep trees to capture potentially complex patterns.

  While this improved the robustness of the Random Forest relative to naive defaults, it still did not surpass Linear Regression on the validation metrics.

6.4 Feature Selection and UI Design

From EDA and domain reasoning, we identified a set of high-impact and user-friendly features to expose in the Streamlit app:

- Overall quality (`OverallQual`)
- Living area (`GrLivArea`)

- Garage capacity (`GarageCars`)
- Basement area (`TotalBsmtSF`)
- Year built (`YearBuilt`)
- Number of full bathrooms (`FullBath`)
- Number of fireplaces (`Fireplaces`)
- Lot area (`LotArea`)

These features strike a balance between:

- **Predictive power** (high correlation with price).
- **User interpretability** in the UI.
- **Simplicity** of input, avoiding overwhelming the user with dozens of sliders and text fields.

6.5 Deployment Optimizations

- **Single artifact:** Packaging the entire preprocessing + model pipeline into one joblib file simplifies deployment and reduces the risk of mismatched transformations.
- **Version pinning:** We pinned critical library versions in `requirements.txt` to ensure compatibility with the saved artifact. For example:
  - scikit-learn 1.6.1
  - joblib 1.5.3
  - numpy 2.0.2
  - pandas 2.3.3
- **Robust prediction function:** The prediction wrapper in the app checks for invalid (non-finite or negative) outputs and raises a clear error if encountered.

7 Streamlit Application

Although not strictly required by the rubric, deployment is an important aspect of the project.

7.1 User Interface

The Streamlit app provides:

- A **prediction card** showing the latest estimated price in USD with nice formatting.
- A set of **sliders and number inputs** for the key features listed in the previous section.
- A **prediction history line chart** plotting previous predictions over runs.

- A **history table** summarising each run, including the chosen feature values and the corresponding predicted price.

The UI is styled to be clean and modern, and it operates without a sidebar, focusing the user's attention on the main interactive elements.

7.2 Prediction Workflow

For each prediction:

1. The app collects user inputs for the selected features.
2. `build_input_df` creates a single-row DataFrame with the same columns as the training data and fills provided values.
3. The loaded pipeline transforms the input and produces a log-space prediction.
4. `predict_usd` applies `expm1` if the target transform flag is set to `"log1p"`.
5. The resulting price is validated (finite, non-negative) and presented to the user.
6. The prediction is appended to an in-memory history list, which updates the chart and table.

This design ensures that the deployed system closely mirrors the training environment and that predictions remain consistent and reproducible.

8 Team Contribution

The project was a collaborative effort. Each team member had clearly defined responsibilities, which are summarised below.

Kavya Katal

- Led **model training** and experimentation in the Jupyter notebooks.
- Designed and implemented the **preprocessing pipelines** using `ColumnTransformer` and `Pipeline`.
- Trained and evaluated the **Linear Regression** and **Random Forest** models.
- Contributed to the **technical report** and documentation of the modelling decisions.

Aabir Sarkar

- Handled **data cleaning and preprocessing**, including handling missing values, column type checks, and dataset sanity validation.
- Implemented the **Streamlit application** logic, including loading the saved artifacts, building input DataFrames, and running predictions.
- Wrote and refined the **LaTeX report**, integrating results from the notebook and codebase.
- Created and organised supporting project assets (e.g., figures and configuration files).

Kushagra Maheswari

- Responsible for **dataset sourcing**, ensuring the correct Ames Housing dataset was obtained and integrated into the project.
- Created the **demo video** showcasing the model and the Streamlit app in action.
- Helped verify that the user interface behaved correctly for different input configurations.

Pratyush Chouksey

- Contributed to **dataset sourcing** and validation of the raw data.
- Assisted in writing parts of the **project report**, especially sections related to data description and EDA.
- Helped cross-check the consistency between the notebook results and the deployed model.

9 Conclusion

In this project, we built a complete property price prediction system, from data preprocessing and model training to deployment in a user-friendly Streamlit interface. Through systematic EDA, we identified key features and addressed data quality issues. Using scikit-learn pipelines with a log-transformed target, we compared a baseline Linear Regression model with a Random Forest Regressor and empirically found that the simpler Linear Regression model achieved superior performance (MAE $\approx$\$14,899, $R^2 \approx 0.93$).

The final deployed model provides fast, stable estimates of house prices and demonstrates the importance of robust preprocessing, careful model selection, and end-to-end reproducibility. Future work could explore additional models (e.g., Gradient Boosting, XGBoost), richer feature engineering, uncertainty quantification, and integration of explainability techniques such as SHAP values.